Object Oriented Programming - 3

Table of Contents

- Input and Output
- Collections and Aliasing
- Generics and Type Parameterization (1)
- Libraries and Testing

Input and Output

We are able to read and write to devices. Within most of the exercises you have constructed so far, you will have used `Console.WriteLine` or `Console.Write`.

However, as with any real-world program, we want to be able to build applications that can interact with our hard drive, over a network or stream. Let's start looking into input and output.

Input and Output - Text and Binary

Very broadly, we will interact with two kinds of types of files/streams.

- Text/Human Readable Data
- Binary Data

While, you can consider the idea of *Text* as just another form of **Binary** data, it is more of a specialisation for the sake splitting how we interpret the two.

Input and Output - Text and Binary

Consider the kind of data stored in the following files.

- `HelloWorld.cs`
- `Cat.jpg`
- `Program.exe`
- `TODO.txt`
- `index.html`
- `win32.dll`

Input and Output

It isn't so obvious and the file extension itself is a bit of a lie, it does however provide a hint for the OS and for the user regarding what application will be able to open it.

- If a `.jpg` is seen, an image viewer is likely to be used
- If a `.txt` is seen, this should be viewable with some text editor

Input and Output - Files

While there is only a certain amount of information you need to understand files and use them. There is more to them than just being an object that is on your hard drive, particularly when it comes to how programs represent files, different ways that files can be represented.

In addition, files are not strictly a concepts which only relates to your hard drive. They can map to memory, networking concepts and devices.

Files and Buffers - Processes

The reason why files are a little more complicated is because of how they relate to other mechanisms. Files are related to processes, when a process launches, there are three file/stream constructs it has open.

- `stdin`
- `stdout`
- `stderr`

While they are not located on your hard-drive, they are mapped to a file which represents a segment in memory.

Files and Buffers - Processes

Whenever a process of ours opens a file either on the hard-disk, memory or something else. The process will keep a table of all **opened** files it has.

Similarly, with stdin, stdout and stderr, these files will be part of the same table. Each entry in that table is called a **file-descriptor**. This is represented as an index.

Files and Buffers - File Modes

When interacting with a file, either for opening, creating, writing or, reading, we need to factor in how we are going to using and specifying the mode we want to use it in. These have been standardised within POSIX (Portable Operating System Interface Standard).

There are three modes that we can use a file in.

- Reading (`r`) - Reads from the beginning of the file, assuming the file exists.
- Writing (`w`) - If a file exists, it will truncate (eliminate the data there) and allow the program to write from the beginning of the file.
- Append (`a`) - It will allow writing from the end of the file.

Files and Buffers - Permissions

File permissions is a tricky topic as it is typically an OS specific thing and while C# is a Microsoft language and originally designed to work on Windows, its usage on Unix-based system has far exceeded its original target.

Similarly, with File Modes, Permissions is also standardised within POSIX. File permissions are three tuples and each tuple constitutes three permission types.

- Other - Any user on the system
- Group - Users from the group that the file belongs to
- Owner - User who owns the file (usually the one that creates it but not always).

Files and Buffers - Permissions

In addition, with each category, each one has the following modes

- Read - Allows processes to read the file
- Write - Allows processes to write to the file
- Execute - Allows the file to be loaded and executed

Why would it be important for a process or operating system to know what files a process has opened?

Why the complication around files?

Input and Output - Abstractions

As part of why it is more complicated, it is because the OS and development environments have built an abstraction to unify different IO mechanisms under a few general IO operations.

The burden on interacting with a device to read or write to it, is simply on a vendor. When a programmer opens a file, down the chain from C# calling `ReadLine`, a syscall `read` will occur which will call driver specific code `ext4_read`.

No, this is not going to be the test!

But gives you insight into why input/output is concept that is easy to understand but the implementation details can be quite deep

Interacting with Files - Text (Reading)

To return to some amount of normalcy, we are just going to start off with some basic interaction of files.

Within C#, you have access to the following types to interact with text files.

- `FileStream` - Lower-level class than most but still fairly high-level, will have primitives like Read, Seek, Write. However, it does require interacting with
- `StreamReader` - The usual class to use when reading files, specifically text. This is because `StreamReader` inherits from `TextReader` class.

Reading from a Text File

```
// Opens and will automatically close
using(StreamReader sr = new StreamReader("Text.txt"))
{
    string? line = string.Empty;

    while((line = sr.ReadLine()) != null)
    {
        // Rest of logic here
    }
}
```

Reading from a Text File - (Without `using`)

```
StreamReader sr = new StreamReader("Text.txt");

string? line = string.Empty;
while((line = sr.ReadLine()) != null)
{
    // Rest of logic here
}

sr.Close(); //Closes the file - Release of resources isn't automatic
```

Reading from a Text File

It is worth noting that the above system is not the only way we may want to process text files. We may want to process them in a particular way as they exhibit a pattern.

Interacting with Files - Text (Writing)

Similarly, if there is a Reader class, there is typically a Writer class. When writing, the `StreamWriter` class is typically the preferred type to use. Similarly, it will have access to `Write` and `WriteLine` methods.

Writing to a Text File

```
using(StreamWriter wr = new StreamWriter("LeaderBoard.txt"))
{
    foreach(Player p in players)
    {
        string player = p.name;
        int score = p.score;
        string formatted = "{0}:{1}".Format(player, score);
        wr.WriteLine(formatted);
    }
}
```

As a bit of an activity, consider how you would read a CSV file along with writing to a CSV file.

What other file types would be a little quirky when writing? A hint here would be any file type that is serialisable or produced by a program

When looking at the methods for either `StreamWriter` and `StreamReader`, you may find that there are some similarities between this and something else you have used?

Stream/Handlers - Resource Acquisition and Disposal

With any stream/handler object itself, it will open a file/resource which ownership is under control by the User, Operating System and Hardware itself. This is noteworthy because it gives a window into another tricky component with files.

Interacting with Files - Binary

When interacting and working with non-text files, we typically have a specification we are interpreting or adhering to. Similarly this can be said with text but the interpretation with Binary has a layer where we are no longer deal with text and its associated patterns but data-representation formats.

Interacting with Files - Binary (Why?)

While at the beginning we discussed particular binary files that were used, such as images and video. These are solid use cases but we also have other forms that we may want represented in this form.

These also include:

- Applications
- Network Protocols
- Debug Dumps
- Encrypted Data

Even between computers, it is not always guaranteed that the binary representation between them is going to be same.

I will note here that while big endian CPU architectures still exist and continue being developed, they are very unlikely to show up for most developers

Reading from a Binary File

Assuming the following structure, we will need to interpret it, knowing its specific structure.

By ensuring we follow this pattern we are the able to guarantee that the data is read correctly. This also goes for writing as well. We have the fields we need to interpret.

- Magic - `int64` - This is a signed integer of 64bits, little-endian encoded
- Name - `char[16]` - Stores a name up to 16 characters
- Date - `int32` - Stores a signed 32bit integer, data represents a timestamp in seconds
- Author - `char[32]` - Author of the file, can store up to 32 characters

What does little-endian and big-endian mean?

Different CPU architectures order their integers and other data types differently. While, naturally we may read a hex value as `0x0A0B0C0D`, the order that a little endian would arrange it would be `[0x0D, 0x0C, 0x0B, 0x0A]` while big endian would order it as `[0x0A, 0x0B, 0x0C, 0x0D]`.

This results in little-endian systems treating the first byte as the most-significant bit (assuming reading left to right), where big-endian systems will treat the last byte as the most significant bit. (`0xA` vs `0xD`).

Reading from a Binary File

Let's just read the header of the binary file.

```
using(FileStream fs = File.Open("imagefile.data", FileMode.Open)){
    using(BinaryReader br = new BinaryReader(fs, Encoding.UTF8, false)) {
        char[] name = new char[16];
        char[] author = new char[32];

        long magic = br.ReadInt64(); //Reads and moves the cursor 8 bytes
        br.Read(name, 0, name.Length); // Reads 16 chars after magic
        int date = br.ReadInt32();
        br.Read(author, 0, author.Length); // Reads 32 chars after date
    }
}
```

The above snippet adheres to the layout of the file. If the file is wrong, we can check it against a test case.

Similarly, it should be noted that, when data is similarly sized and bounded by a number of entries within it, we can simply loop through it.

Writing to a Binary File

Once again, adhering to the specification is necessary to ensure interoperability between files. Assuming we have `Header` object which contains the necessary data to write.

```
using(FileStream fs = File.Open("imagefile.data", FileMode.Create)) {
    using(BinaryWriter bw = new BinaryWriter(fs, Encoding.UTF8, false)) {
        Header header = data.Getheader();

        bw.Write(header.Magic);
        bw.Write(header.Name, 0, header.Name.Length);
        bw.Write(header.Date);
        bw.Write(header.Author, 0, header.Author.Length);
    }
}
```

Collections and ADTs

You will be hearing the word **type** a lot in this course and outside of it. As established prior with classes, a class is an abstract datatype. It is an aggregation of other types and primitive types.

Collections

Our applications will require certain needs in storing and organising data. Collections or aggregate types allow us to store data in the appropriate objects.

For example:

- A school contains students
- A stage has performers
- Running times from athletes for a race
- TV Shows on a streaming platform

Collections

The .NET standard library provides a standard set of collections that you can use and have probably used prior such as:

- List types
- Set types
- Map types

Collections - List Types

The usual list type that is used is a called **List**.

It is somewhat vague in how it is actually structured but we are aware that it is able to keep adding and resize on demand.

There are other list types:

- `LinkedList`
- `Queue`
- `Stack`

However, depending on how we store our data, update, and access the data will determine what type we want to use. Similarly for set and map types.

Collections - List

Let's consider the following problem and see if we can solve it by just using arrays.

> We want to store all input that the user will provide via keyboard to our program, the amount of input is not known beforehand.

Hmm, let's consider a problem here, they are asking us to store **all input** and that it is **not known beforehand**.

We have a problem, arrays are fixed length, so... how?

A solution exist but we have to do some work.

Scenario - Reading input and storing all of its

Let's walk through a solution that will keep reading until the user is finished.

```
    //capacity and size snipped
    string[] allLines = new string[capacity];
    string? line = Console.ReadLine();
    while(line != null) {
        if(size >= capacity) { // Check if we need to resize
            capacity = capacity * 2;
            string[] newAllocation = new string[capacity];
            for(int i = 0; i < size; i++) {
                newAllocation[i] = allLines[i]; //Copy over
            }
            allLines = newAllocation; // Drop old allocation and assign new one
        }
        allLines[size] = line;
        size++;
        line = Console.ReadLine();
    }
```

Collections - List

Okay, but what does this have to do with lists?

Well, we had our constraint which was to not use a collection that would do this for us, we have now
found a way to achieve a similar effect but with Arrays.

What is being inferred here is that we can build a collection `List` using the same idea.

- List elements are retrievable via an index
- Lists can have new data added
- Lists can resize

Collections - List

As with most collections that we can use from the standard library and other libraries, these libraries will usually require a type to be provided.

```
List<string> lines = new List<string>();
```

Collections will typically require you to specify the type that is contained, this is called providing a type argument. Where it specifies `<string>`, between the two angle brackets we are specifying the type.

Collections - List

More formally, our `List`, is typically called a `Dynamic Array`, this is where we have the following properties with our list.

- Capacity
- Size
- Array where we can store the elements

The capacity will usually be set to a non-zero value, outlining the number of places that can be used.

Size is used to indicate the number of elements we have stored inside it.

Collections - Maps and Sets

We want to be able to use non-integer objects for storage. This where we bring in two different types that allow this: **Maps** and **Sets**.

Within C#, a **Map** type is usually called a **Dictionary**, outside of C# this is usually just referred to has **Map** or **HashMap** to indicate its particular implementation.

Sets within C# as called **HashSet**, this is a little unusual due to their naming conventions but does hint to the implementation of the **Set** collection itself.

Collections - Map

Map and set types also utilise type parameters that, **Set** similarly to **List** which is a single type parameter.

```
Set<int> numbers = new Set<int>();
Dictionary<string, string> addresses = new Dictionary<string, string>();
```

However, the **Map** type which outlines a `TKey` and `TValue` type parameter, the **K** parameter within Map specifies the `IComparable`. This is the same with **Set**, which only has **unique** elements.

Why do we need to specify `IComparable` for key types?

This is because keys in both **Set** and **Map** are required to be comparable to ensure uniqueness. Without this constraint being enforced, retrieval and/or insertion would be considered unsound.

Collections - Set Usage

When using a **HashSet** within C#, we may want to identify an appropriate use-case for it. This could be something where we are keeping track of elements found between two **Lists**.

```
HashSet<string> foundWords = new HashSet<string>();
foreach(string word1 in wordlist1) {
    string foundWord = string.Empty;
    if(!foundWords.TryGet(word1, out foundWord)) { // Not FOund
        foreach(string word2 in wordlist2) {
            if(word1 == word2) {
                foundWords.Add(word1);
                break;
            }
        }
    }
}
```

Collections - Set Usage

The snippet below is where we are identifying if we have found a word already and if it can be skipped or if we have to search and add it to the set.

```
if(!foundWords.TryGet(word1, out foundWord)) // Not found
{
    foreach(string word2 in wordlist2)
    {
        if(word1 == word2) {
            foundWords.Add(word1);
            break;
        }
    }
}
```

Collections - Map/Dictionary Usage

Dictionary usage is a lot more prevalent, normally because we either lack a quick function to compute the result for a particular set of input or we need to creating a mapping from one set to another.

A common use-case would be address books, far-be it the only use-case though.

```
Dictionary<string, string> addresses = new Dictionary<string, string>();
addresses.Add("Jeff", "jeff@jeff.co");
addresses.Add("Alice", "alice@gmail.co");
addresses.Add("Bob", "bob@protonmail.com");
```

Also, to note on accessing element, while `TryGet` is what I would favour, you can use `[key]` as a way to get the element, however this may result in an exception if the result is not present.

Generics

Generics are specified as part of the class definition. We are able to incorporate parameter types that can be used for specifying the data types for fields, properties, and methods.

```
class Box<T>
```

The above demonstrates the specification of the type parameter `T`. The developer can then specify the type argument when using this type.

Generics

Once we have a symbol that will represent the type through out the class, we can use it throughout. While it is specified along with the class, it can be used with static methods as well that then allows the user to input the type per-method usage.

Examples.

```
public T GetData();

public void SetData(T data);

public void ProcessData<T>(T[] data);

public T StartValue { get; set; }
```

Generics - Container

Let's start off with constructing a symbol container that will hold any kind of object.

```
class Container<T>
{
    public T? Data { get; set; } = null;

    public Container(T? element)
    {
        Data = element;
    }

    public bool IsNull()
    {
        return Data == null;
    }
}
```

Generics - Container

We can observe the property which is set to null but can be set to a value will is using the type parameter as its datatype.

```
    public T? Data { get; set; } = null;
```

Generics - Container

The constructor for the datatype is expecting an argument which matches type T, T has to be specified as part of the initialisation of the object.

```
    public Container(T? element)
    {
        Data = element;
    }
```

Generics - Container

Method to simply check if it is null or not. Nothing to spectacular but demonstrates how it is able to be used.

```
    public bool IsNull()
    {
        return Data == null;
    }
```

Generics - Container (Usage)

Let's now use this class within a simple code snippet and understand what we need to do and how this works.

```
Container<string> stringBox = new Container<string>("Hello World!");
```

We can observe that we have had to specify `<string>` as part of declaration type and the initialisation. This is to ensure the binding type and object it is allocated to are compatible as Generics are a little stricter than what we may be use to.

Generics - Container (Usage)

Let's map what is occurring as well. We can see the type argument as input for the class `Container`. This is what `T` will be representing for this instance.

```
Container<string> stringBox = new Container<string>("Hello World!");
```

So, where `string` or other types are used, `T` will be represented as such.

```
class Container<T>
```

Class Library

When developing software we will make many different components through out our time which we will be able to reuse. A common method within C# and other programming languages is to create a library. Specifically in C#, a class library.

Why would we want to do this?

- Reusing code
- Increasing portability
- Reducing workload
- Clear scope boundaries of work
- Modularising and making components

Class Library

While we will be brief as building a class library is nothing too substantial. It is best to be exposed to the idea on how to make one and use it in another project.

```
dotnet new classlib -n "NewIO"
```

The above will create a new class library called `NewIO`.

Class Library

The class library `NewIO`, can be then referenced by another solution. A common method is to do the following.

```
dotnet add FastServer.csproj reference ../NewIO
```

The above snippet will reference `NewIO` within the `FastServer` project. This will allow `using` the namespaces that `NewIO` exposes.

In regards to encapsulation, this is where anything that is considered `internal` will not be exposed to the `FastServer` project but anything public will be.

Testing and Using `xUnit`

We now enter into our first foray of unit-testing. While it is convention to separate logic into applications and libraries, it is possible to use xunit with just an application itself.

Similar to our classlib example, we are able to create a project using `xunit` as the template name.

Testing and Using `xUnit`

You will utilise the xunit template with dotnet which will setup and download the necessary components for your project. As per convention, the naming of the testsuite will have the suffix of `.Test`.

```
dotnet new xunit -o Palindrome.Tests
```

Testing and Using `xUnit`

We will use the Palindrome task as a way to present the usage of xunit. In this case we will use a `classlib`. One of the main classes and methods is going to be the following.

```
namespace Palindrome;

public class Palindrome
{
    public static bool IsPalindrome(string line)
    {
    // Code here!
        return false;
    }
}
```

Testing and Using `xUnit`

Given this static method that is associated with Palindrome, we will now update `UnitTest1.cs` to be `PalindromeTest.cs` for clarity sake.

```
public class PalindromeTestSuite
{
    [Fact]
    public void IsPalindomre_Test_ABBA()
    {
        Assert.True(Palindrome.IsPalindrome("abba"));
    }
```

`Fact` is an attribute that is placed above the method and declares the method a particular test case. Within each method, you can test particular values with an expected outcome using methods associated with Assert

Testing and Using `xUnit`

Once we have laid out the test cases we want to go through, we are then able to call `dotnet test`. This will trigger the test loader for `xunit` to run.